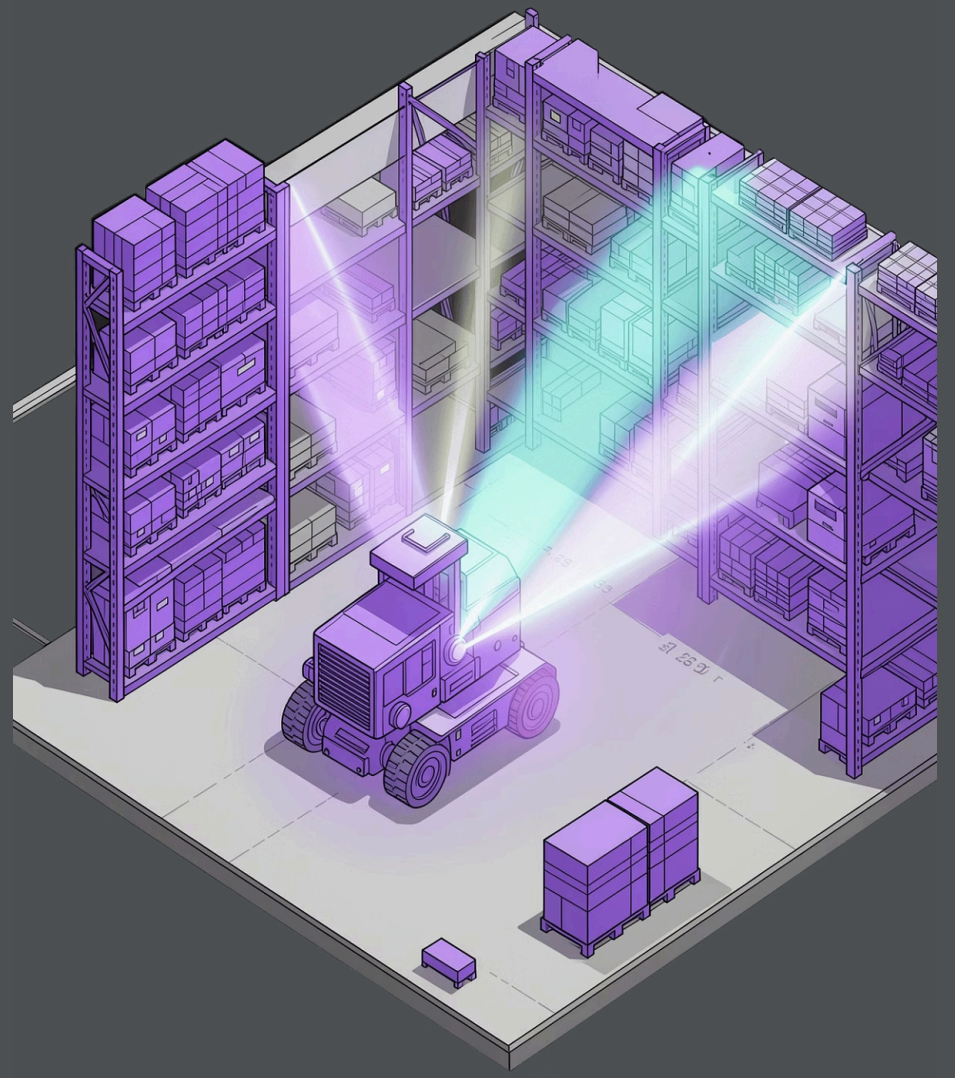
Nova Carter Perception Pipeline

Isaac Sim + ROS 2 + YOLOv8 — Technical Progress Review

PERCEPTION ENGINEERING | IN PROGRESS



Made with GAMMA

Project Objective

Mission Statement

Develop a fully validated perception pipeline for the Nova Carter mobile robot platform running inside **NVIDIA Isaac Sim**, leveraging **ROS 2** as the middleware backbone and **YOLOv8** as the inference engine for real-time object detection.

Current Phase

RGB object detection in a lightweight, GPU-optimized flat grid test scene. Camera pipeline validated, YOLO inference live.

Future Phase

RGB–Depth–LiDAR sensor fusion for full 3D spatial awareness in warehouse robotics deployments.

End-to-End System Flow

01

Isaac Sim

Physics-accurate simulation environment

02

Nova Carter Sensors

Front RGB camera + future depth/LiDAR

03

ROS 2 Middleware

Topic-based message transport

04

Perception Nodes

nova_carter_perception package

05

YOLOv8 Inference

Object detection + output publication

06

Depth/LiDAR Fusion

3D position estimation (next milestone)

Made with GAMMA

Development Environment & Stack

Full-stack configuration spanning hardware constraints, robotics middleware, and AI inference libraries.

Hardware & OS

- Ubuntu 24.04 LTS
- ROS 2 Jazzy
- Python 3.12
- NVIDIA Isaac Sim
- RTX 3050 GPU — **4 GB VRAM**

ROS / Robotics

- `rclpy` — Python client library
- `sensor_msgs` — Image message types
- `cv_bridge` — ROS ↔️ OpenCV bridge
- `vision_msgs` — Detection2DArray
- ROS 2 QoS profiles for reliable comms

AI / Computer Vision

- Ultralytics **YOLOv8n** (nano variant)
- PyTorch — inference backend
- OpenCV — image processing
- NumPy — array operations

YOLOv8n (nano) was selected to minimize GPU memory footprint on the 4 GB RTX 3050 while maintaining adequate detection accuracy for proof-of-concept validation.

Made with GAMMA

Simulation Optimization for Limited GPU

The Challenge

The **RTX 3050 (4 GB VRAM)** produced **Vulkan out-of-device-memory** errors when running complex scenes with multiple RTX-enabled sensors simultaneously. A systematic optimization strategy was required to achieve a stable simulation baseline.

Result

Stable, repeatable simulation environment enabling consistent perception pipeline validation — all GPU constraints resolved prior to AI integration work.

Optimization Decisions

Environment Swap

Replaced heavy warehouse scene with a lightweight **Flat Grid** scene — dramatically reduced rendering overhead.

Robot Selection

Deployed **Nova_Carter_ROS** for its native ROS 2 integration, eliminating custom bridge development.

Sensor Reduction

Enabled **only** the front RGB camera at **640 × 360** resolution. All other sensors disabled.

Disabled Components

Fisheye cameras, depth pipeline, unused cameras, and supplementary LiDAR units all turned off.

Made with GAMMA

System Architecture

End-to-end data flow from Isaac Sim sensor output through the custom `nova_carter_perception` ROS 2 package to standardized detection outputs.



Node: `rgb_listener`

Subscribes to the front stereo camera topic and performs camera-to-ROS bridge validation. Decodes `sensor_msgs/Image` messages via `cv_bridge` and saves raw frames locally. Serves as the isolated communication health check before any AI inference is added to the pipeline.

Node: `yolo_detector`

Consumes the live camera topic, converts frames through `cv_bridge`, and feeds them continuously into YOLOv8n. Extracts class labels, confidence scores, and bounding-box coordinates per frame. Publishes both annotated image output and structured detection messages to downstream nodes.

Made with GAMMA

Milestone 1: Camera Pipeline Validation

Objective

Validate the ROS 2 camera pipeline in isolation — **before** introducing any AI inference layer. This deliberate sequencing separates ROS communication failures from model-level failures, making debugging tractable.

Validated Pipeline

1. Isaac Sim RGB Camera
2. sensor_msgs/Image
3. rgb_listener Node
4. cv_bridge → OpenCV
5. Saved Frame

Confirmed Achievements

Topic Reception

Nova Carter front camera topic successfully received with zero message loss over extended run periods.

Resolution Confirmed

Camera operating at **640 × 360** pixels — optimized for 4 GB VRAM constraint.

Continuous Stream

Sustained image frame delivery through ROS 2 middleware confirmed at expected rates.

Frame Capture

First RGB frame captured and persisted locally — used as input to Milestone 1.5 offline inference.

Engineering principle confirmed: isolating the transport layer from the inference layer is a proven debugging strategy that paid dividends in this project.

Made with GAMMA

Milestone 1.5: YOLO Proof of Concept

Offline validation of YOLOv8n on a captured Isaac Sim frame — confirming model compatibility and inference quality **before** live ROS integration.

Test Pipeline

01

Saved RGB Frame

Captured from Milestone 1 camera validation

02

YOLOv8n Inference

Offline execution — no ROS dependencies

03

COCO Class Detection

Person class from 80-class COCO dataset

04

Output Verification

Bounding box, class label, confidence score

Validated Results

Detection Confidence

Person detected at **~0.93** confidence on first offline inference run against an Isaac Sim-rendered frame.

Model Compatibility

YOLOv8n loaded without errors. Isaac Sim RGB output confirmed fully compatible with Ultralytics input format.

Bounding Box Output

Bounding-box generation and COCO class identification both confirmed correct on simulated scene imagery.

Confidence Estimation

Confidence scoring pipeline functioning correctly — output values consistent with expected model behavior.

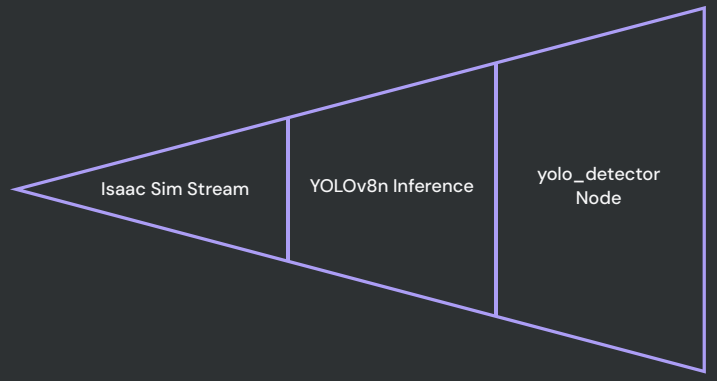
Milestone 1.5 de-risked live integration by confirming all model-level unknowns offline first.

Made with GAMMA

Milestone 2: Live ROS + YOLO Integration

Direct connection of YOLOv8n inference to the live Nova Carter ROS 2 image stream — the core engineering achievement of the current phase.

Live Pipeline



yolo_detector Node — Responsibilities

- Subscribes to live front RGB camera topic from Isaac Sim ROS bridge
- Converts `sensor_msgs/Image` to OpenCV array via `cv_bridge`
- Performs continuous frame-by-frame YOLOv8n inference
- Extracts class label, confidence score, and bounding-box coordinates per detection
- Generates annotated image overlays and publishes structured detection messages

Performance Metrics

0.84–0.90+

Confidence Range

Stable person detection across varied Isaac Sim scene conditions

~10s ms

Inference Latency

Tens of milliseconds per frame — sufficient for current camera stream rate

640×360

Input Resolution

Matched to GPU-optimized camera configuration

Live integration confirmed that the isolated validation approach (Milestones 1 and 1.5) was highly effective — no ROS/model compatibility issues arose during live hookup.

Made with GAMMA

Standardized ROS Outputs

The yolo_detector node publishes two standardized ROS 2 topics, enabling clean separation between visualization and downstream algorithmic consumers.

Topic 1: /yolo/annotated_image

Message Type

sensor_msgs/Image

Contents

RGB image with overlaid YOLO bounding boxes, class labels, and confidence scores rendered directly into the pixel buffer.

Primary Use

Human-readable visualization and pipeline debugging. Consumable by RViz2 or any standard image viewer node.

Topic 2: /yolo/detections

Message Type

vision_msgs/Detection2DArray

Contents

Timestamp, object class, confidence score, bounding-box center coordinates, width, and height — fully structured per detection.

Example Payload

Class: **person** | Confidence: **0.86** | BBox: 2D image coordinates (center x, center y, w, h)

Primary Use

Machine-readable input for depth estimation, object tracking, and sensor fusion nodes — fully reusable by Milestone 3 architecture.

The vision_msgs/Detection2DArray schema is a ROS 2 standard — downstream fusion nodes can consume /yolo/detections without any dependency on the YOLOv8 implementation internals.

Made with GAMMA

Milestone 3: Depth & LiDAR Fusion

Capability Leap

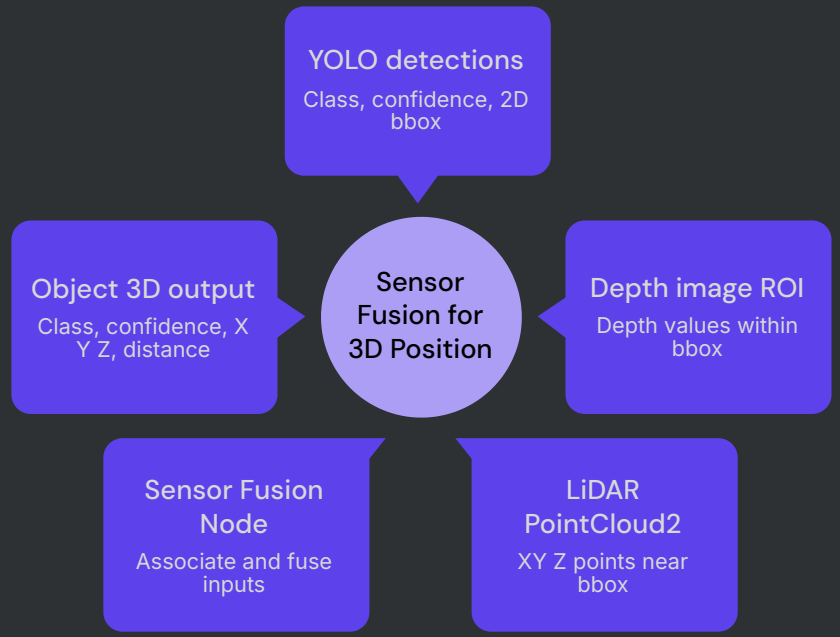
Today:
"There is a person in this image."

After Milestone 3:
"There is a person approximately **X meters** from the robot at 3D position (X, Y, Z)."

Target Output Format

Person — Confidence **0.86** — Distance **3.2 m** — Position **(X, Y, Z)**

Sensor Fusion Architecture



Planned Inputs

Source	Topic / Type	Data Provided
YOLOv8	/yolo/detections	Class, confidence, image bounding box
Depth Camera	sensor_msgs/Image	Per-pixel depth within detected region
LiDAR	sensor_msgs/PointCloud2	X, Y, Z point cloud + distance measurement

Made with GAMMA